CS-781 Capstone, Phase 4: Court Judgment Prediction

A hierarchical chunk classifier on the four Phase-1 language models

Manish Kumar (251330604)
CS-781, IIT Kanpur

October 2026

Abstract

We predict the binary outcome of Indian court judgments from their text, starting from the four legal language models pre-trained in Phase 1. A judgment is cut into 512-token chunks; eight chunks are encoded by the Phase-1 backbone, mean-pooled, read by a BiGRU with attention pooling and classified. Eight experiments cover all four architectures and seven adaptation techniques (full and partial fine-tuning, gradual unfreezing, BitFit, LoRMA+, AdaLoRA and Houlsby adapters) plus knowledge distillation. Because every test document also appears, labelled, in the provided development file, all model selection uses only the 1,988 development documents that are not in the test set. On that clean validation set the best single model (A4, top four blocks fine-tuned) reaches macro-F1 0.785, the distilled 6-block student 0.776 (98.8% of its teacher with 62.8M instead of 88.0M parameters), and an average of five models 0.797. Reading the last chunks of a judgment scores higher than reading the first (0.785 vs 0.754). All numbers are generated from the run outputs by `p4/report/make_figures.py`.

Contents

1 Introduction **2**

2 Data and integrity checks **2**

3 Methods **3**
3.1 Hierarchical chunk model . . . 3
3.2 Adaptation techniques . . . 3
3.3 Training and selection . . . 4
3.4 Submissions . . . 4

4 Results **4**
4.1 All experiments . . . 4
4.2 Decision threshold and class imbalance . . . 6
4.3 Which part of the judgment . . . 6
4.4 Knowledge distillation and efficiency . . . 6
4.5 Ensembles . . . 7
4.6 Errors of the best single model . . . 7
4.7 Submission files . . . 7

5 Analysis **8**

6 Limitations and next steps **8**

7 Conclusions **8**

8 Reproducibility **9**

1 Introduction

Court judgment prediction (CJPE) asks for the outcome of a case (label 1: the appeal is allowed or the order set aside; label 0: dismissed) from the judgment text, with the final decision removed; the data follow the ILDC corpus [7] (single- and multi-instance training sets combined). The primary metric is macro-F1 over the two classes, with precision and recall as secondary metrics. Judgments are long (median 2,469 BPE tokens in training, 3,771 in validation), so the model must represent a long document. The rules require the Phase-1 models as the only pretrained weights, all four architectures across the submissions, at least six substantially different experiments with at least five adaptation techniques, LoRMA and knowledge distillation.

2 Data and integrity checks

Table 1: Splits used (lengths in Phase-1 BPE tokens).

Split	Documents	Label 0 / 1	Median length	90th pct.	Tokens
train	31,677	18,552 / 13,125	2,469	7,795	121.5M
val	1,988	994 / 994	3,771	9,239	10.0M
test	1,517	– (not read)	3,932	9,639	7.8M

Findings in the provided files. Three checks were made before any training (`p4/cj/data.py`):

- `test.json` carries a `label` field for every test document (yes). It is dropped when the file is loaded and never read.
- All 1,517 of the 1,517 test documents occur verbatim, with their labels, in `dev.json` (3,505 documents). Selecting models or thresholds on the full development file would therefore tune on the test answers. We use as validation only the 1,988 development documents whose text is not in the test set (994 / 994 labels).
- The training file has 37,387 entries but only 31,677 distinct texts (5,710 exact duplicates, none with conflicting labels). Duplicates are removed.

Training is 58.6% label 0, while validation (and the test set, by its size) is balanced.

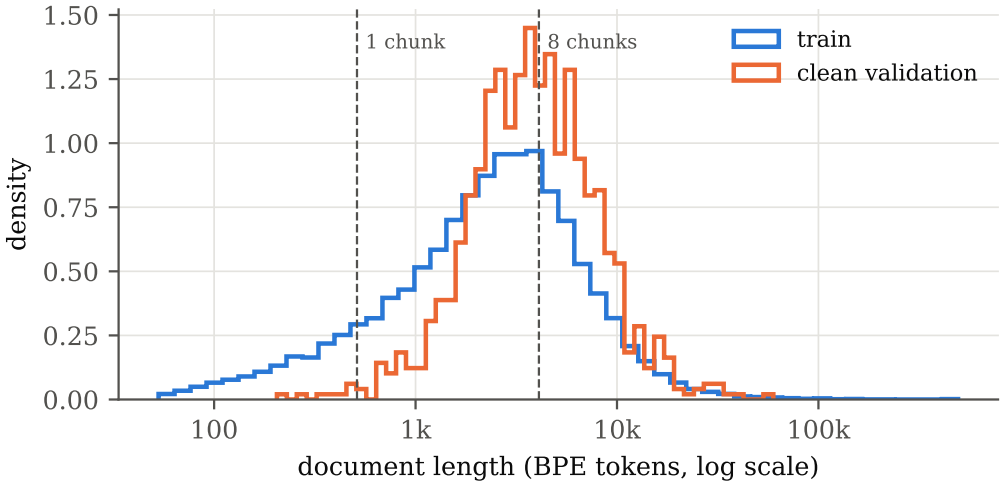


Figure 1: Document lengths. Eight 512-token chunks cover the whole document for 55% of the validation documents.

3 Methods

3.1 Hierarchical chunk model

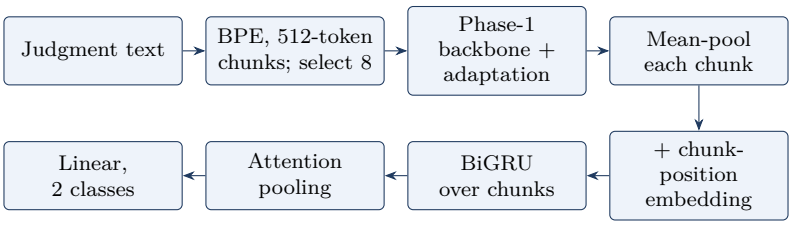


Figure 2: The CJPE model (`p4/cj/model.py`).

- **Chunking** (`data.strategy`). The document is tokenized once and cut into chunks of 512 tokens; eight are kept: the *last* eight (default; aligned to the end of the document), the *first* eight, *first four + last four*, or eight *spread* evenly. Shorter documents use all their chunks.
- **Chunk vector**: the mean of the backbone's final hidden states over the chunk's tokens.
- **Document layer**: projection to 256 dimensions plus a learned chunk-position embedding, a one-layer BiGRU [3] over the chunk sequence and attention pooling with a learned scoring vector (hierarchical attention, [8]); a linear layer gives two logits.
- **Backbones**: A1 and A4 keep their Phase-1 causal mask (best in Phase 3), A3 reads each chunk as its bidirectional prefix, A2 uses its decoder states (its encoder output is nearly collapsed, Phase-2 report). All use the cleaned 1B-token Phase-1 checkpoints.

3.2 Adaptation techniques

- **Full fine-tuning** (E1): every weight.
- **Partial / layer-wise fine-tuning** (E6): the top four of A4's ten blocks and the final norm.
- **Layer freezing / unfreezing** (E7): ULMFiT-style gradual unfreezing [6]; only the top block (and final norm) is trainable at the start, one more block is unfrozen every 300 optimizer steps, top to bottom, until the whole network trains.
- **BitFit** (E4) [1]: biases and norm gains only.
- **LoRMA+** (E2) [2]: $W = (I + \frac{\alpha}{r} BA)W_0$, $r = 16$, $\alpha = 32$, on the attention and MLP projections.
- **AdaLoRA** (E5) [9]: SVD-form update with importance-based rank pruning from 12 to an average of 8 per module (implementation of Phase 3).
- **Houlsby adapters** (E3) [5]: bottleneck 64 after attention and MLP.
- **Knowledge distillation** (E8) [4]: the teacher is E6 (best single model); its logits on all 31,677 training documents are computed once (`dump_logits.py`). The student is A4 with 6 of its 10 blocks (0, 2, 4, 5, 6, 8, keeping all three global-attention layers), fully fine-tuned with $0.5\,\mathrm{CE} + 0.5\,T^2\,\mathrm{KL}(p^T_{\text{teacher}} \,\|\, p^T_{\text{student}})$, $T = 2$.

3.3 Training and selection

AdamW, 2 epochs over the 31,677 training documents (3,960 optimizer steps of 16 documents, 4 per micro-batch), 6% warm-up then cosine decay, weight decay 0.01, gradient clipping 1.0, bf16. Learning rate 5×10^{-5} for full fine-tuning, gradual unfreezing and the KD student, 10^{-4} for partial fine-tuning, 2×10^{-4} for LoRMA+, adapters and AdaLoRA, and 10^{-3} for BitFit and the document layer. The PEFT runs were first started at 10^{-3} (the Phase-2/3 value); LoRMA+ and adapters stalled at a training loss of 0.69 (chance) within the first 1,000 steps and were restarted at 2×10^{-4}, which is the run reported here. The loss is class-weighted cross-entropy (weights inverse to the training label frequencies). Every 250 steps the clean validation set is scored; the state with the best macro-F1 at the validation-tuned decision threshold (grid 0.05–0.95) is kept, and that threshold is applied to the test set.

3.4 Submissions

`make_submission.py` writes `prediction.json`, a list of `{"id", "pred"}` with one integer 0/1 for every test id in order, at the root of a zip. Ensembles average the class-1 probabilities of several runs and tune one threshold on their averaged validation probabilities.

4 Results

4.1 All experiments

Table 2: Clean-validation results (1,988 documents). F1 = macro-F1 at the validation-tuned threshold "thr"; F1@0.5 at the default threshold; P / R = macro precision / recall at thr. Below the line: section ablation of E6.

Run	Arch	Method	Chunks	Trainable	%	F1	F1@0.5	thr	P	R
E1_a1_full	A1	full fine-tuning	last	82.58M	100.0	0.7698	0.7398	0.42	0.775	0.771
E2_a4_lorma	A4	LoRMA+	last	2.72M	3.0	0.7535	0.7325	0.35	0.754	0.754
E3_a3_adapter	A3	Houlsby adapters	last	2.12M	2.5	0.7541	0.7211	0.41	0.756	0.755
E4_a2_bitfit	A2	BitFit	last	0.60M	0.7	0.6614	0.5330	0.29	0.663	0.662
E5_a1_adalora	A1	AdaLoRA	last	1.84M	2.2	0.6001	0.4368	0.40	0.603	0.602
E6_a4_partial	A4	partial (top 4 blocks)	last	25.68M	29.2	0.7854	0.7502	0.39	0.787	0.786
E7_a3_gradual	A3	gradual unfreezing	last	81.79M	100.0	0.7726	0.7228	0.31	0.773	0.773
E8_kd_a4_small	A4	KD student (full FT)	last	62.85M	100.0	0.7756	0.7666	0.56	0.776	0.776
A_e6_first_last	A4	partial (top 4 blocks)	first_last	25.68M	29.2	0.7785	0.7717	0.47	0.780	0.779
A_e6_spread	A4	partial (top 4 blocks)	spread	25.68M	29.2	0.7776	0.7541	0.43	0.778	0.778
A_e6_first	A4	partial (top 4 blocks)	first	25.68M	29.2	0.7535	0.6986	0.34	0.754	0.754

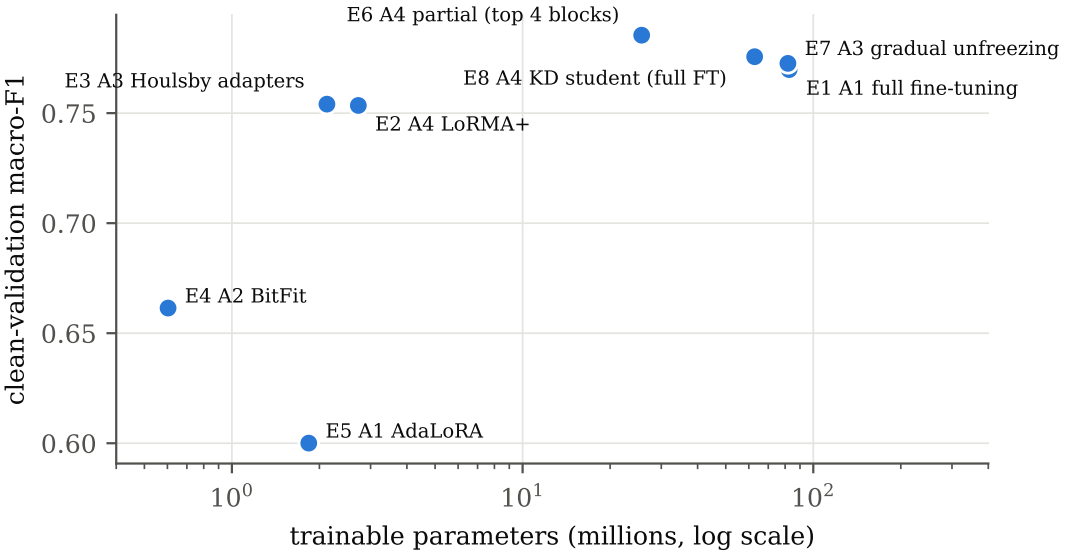


Figure 3: Validation macro-F1 against the number of trained parameters (document layer included).

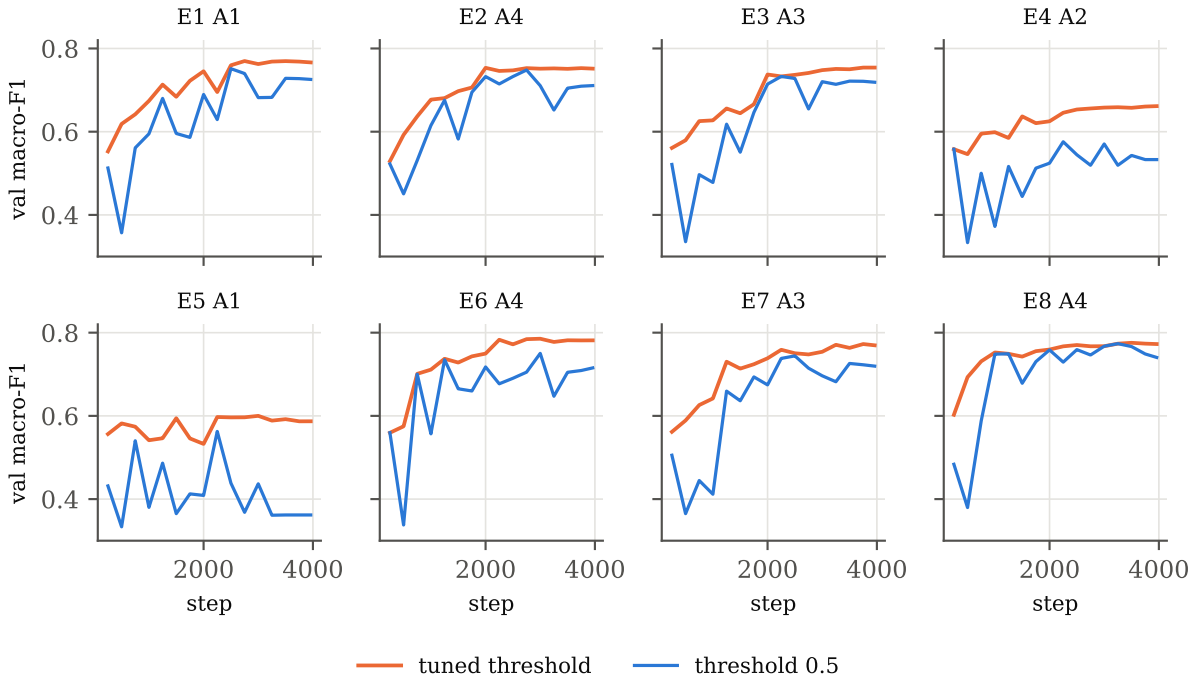


Figure 4: Validation macro-F1 during training, at the default and at the tuned threshold.

4.2 Decision threshold and class imbalance

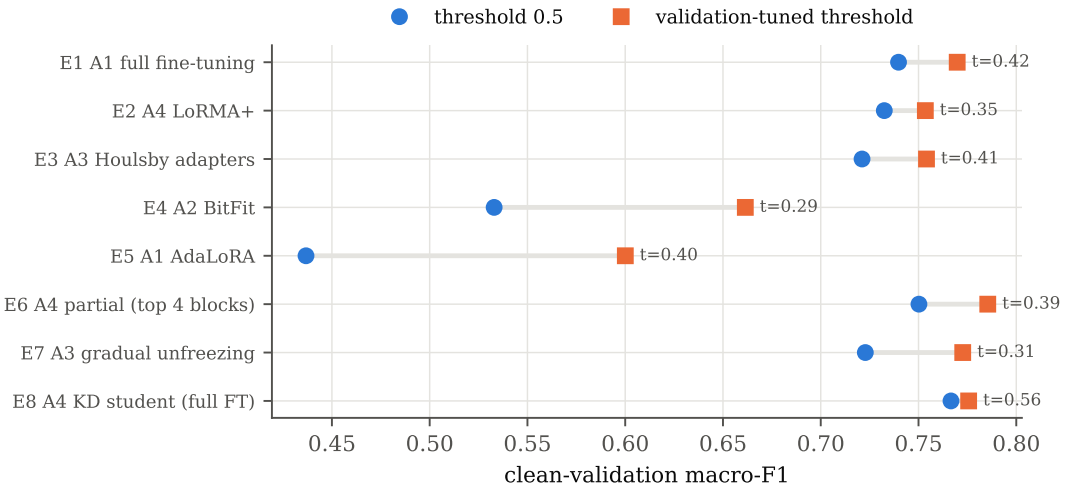


Figure 5: Macro-F1 at threshold 0.5 and at the validation-tuned threshold t (training is 58.6% label 0, validation 50%).

4.3 Which part of the judgment

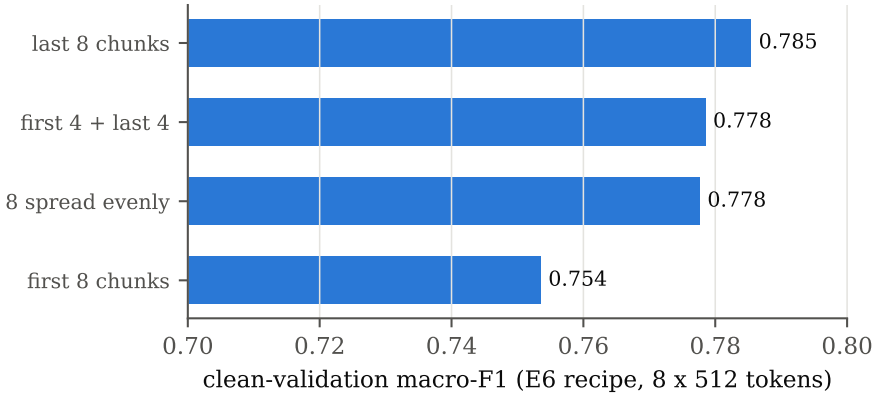


Figure 6: Section ablation: E6 recipe with different chunk selections (same 8 × 512 tokens).

4.4 Knowledge distillation and efficiency

Table 3: Size, compute and memory. Test inference = seconds to score the 1,517 test documents.

Run	Arch	Method	Total	Trainable	Train (min)	Test infer. (s)	Peak GB	Ckpt MB
E1	A1	full fine-tuning	82.6M	82.58M	41.6	24.9	5.9	330.3
E2	A4	LoRMA+	90.2M	2.72M	68.6	13.1	8.3	10.9
E3	A3	Houlsby adapters	83.4M	2.12M	45.5	21.5	3.7	8.5
E4	A2	BitFit	89.9M	0.60M	50.2	28.7	4.7	2.5
E5	A1	AdaLoRA	83.9M	1.84M	71.4	7.1	5.9	7.4
E6	A4	partial (top 4 blocks)	88.0M	25.68M	47.6	20.5	4.0	102.7
E7	A3	gradual unfreezing	81.8M	81.79M	27.7	9.2	5.6	228.9
E8	A4	KD student (full FT)	62.8M	62.85M	34.8	5.8	5.9	251.4

The KD student (E8) has 62.8M parameters against 88.0M for its teacher E6 and keeps 98.8% of the teacher's validation macro-F1; test inference takes 6 s against 20 s.

4.5 Ensembles

Table 4: Probability-averaging ensembles (threshold tuned on the averaged validation probabilities).

Name	Members	F1	thr	P	R
ENS_top3	E6, E7, E1	0.7931	0.42	0.804	0.795
ENS_top5	E6, E7, E1, E3, E2	0.7974	0.39	0.800	0.798

4.6 Errors of the best single model

E6 makes 176 false positives (predicted allowed, actually dismissed) and 250 false negatives on the validation set.

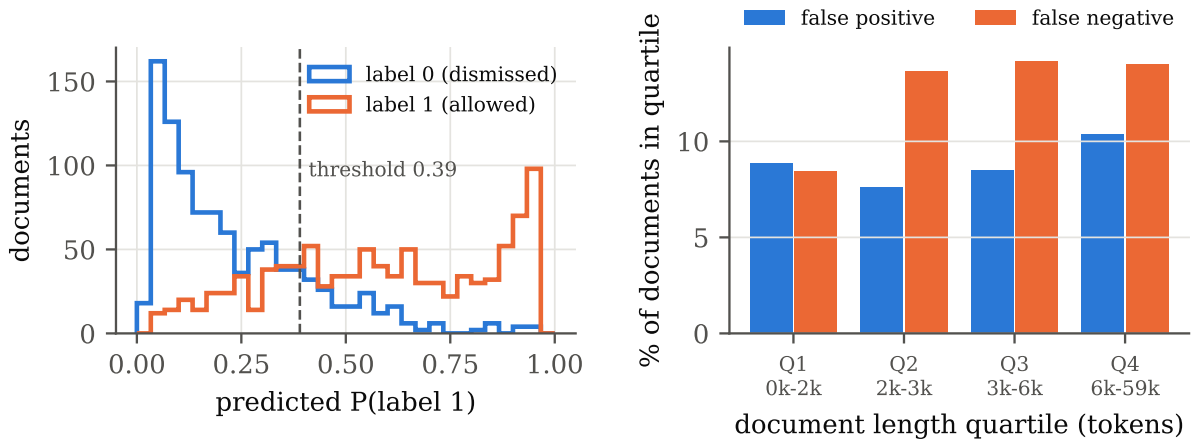


Figure 7: E6 on clean validation. Left: predicted probability of label 1 per true class. Right: false-positive and false-negative rate by document-length quartile.

Table 5: E6 by document-length quartile (tokens).

Quartile	Length	Docs	Macro-F1	FP	FN
Q1	205–2,413	496	0.826	44	42
Q2	2,413–3,771	498	0.786	38	68
Q3	3,771–6,087	494	0.771	42	70
Q4	6,087–59,715	500	0.755	52	70

4.7 Submission files

16 format-checked zips are in `p4/submissions/` (an experiment with two zips has identical predictions in both).

To be written by the author — leaderboard scores

Add the Codabench test macro-F1, precision and recall of each uploaded submission and say which six or more count towards the requirement.

File
`A_e6_first_20261017_135437.zip`
`A_e6_first_last_20261017_135448.zip`
`A_e6_spread_20261017_135456.zip`
`E1_a1_full_20261016_235009.zip`
`E1_a1_full_20261017_002609.zip`
`E2_a4_lorma_20261017_002336.zip`
`E2_a4_lorma_20261017_002617.zip`

File
E3_a3_adapter_20261017_000036.zip
E3_a3_adapter_20261017_002625.zip
E4_a2_bitfit_20261017_002632.zip
E5_a1_adalora_20261017_002640.zip
E6_a4_partial_20261017_002647.zip
E7_a3_gradual_20261017_002655.zip
E8_kd_a4_small_20261017_135920.zip
ENS_top3_20261017_131750.zip
ENS_top5_20261017_131759.zip

5 Analysis

To be written by the author — analysis of the results

The course rules do not allow LLM-written result analysis. Questions the course asks (§10.5), with the evidence above:

- Which adaptation techniques are effective, and can parameter-efficient methods retain full fine-tuning performance (Table 2, Figure 3)?
- How do long-document representations and chunk selection affect prediction (Figure 6)?
- Which sections are useful, based on the ablation?
- How does class imbalance affect performance (Figure 5)?
- What characterises false positives and false negatives (Figure 7, Table 5)?
- Does distillation keep performance while reducing size and compute (Table 3)?
- How well does the Phase-1 model transfer to CJPE?

6 Limitations and next steps

Facts that limit the conclusions (the interpretation is the author's):

- One seed per run; the validation set has 1,988 documents and the threshold is tuned on it, so the reported F1 is slightly optimistic.
- Eight chunks cover the whole document for only 55% of validation documents.
- AdaLoRA (E5) and BitFit (E4) were run with one setting each.
- The PEFT learning rate was changed once during the phase (Section 3.3).
- The provided test labels and the test/dev overlap (Section 2) mean the public leaderboard may not reflect held-out performance for every participant.

To be written by the author — what you would do next

With more time or compute: ...

7 Conclusions

To be written by the author — conclusions

Summarise what you conclude from the experiments, in your own words.

8 Reproducibility

Token cache	built on first use by `cj/data.py` (`build_cache`)
Experiments E1–E7 + zips	`bash run_all.sh`
One experiment	`python train.py -config configs/e6_a4_partial.yaml`
Teacher logits + KD	`python dump_logits.py -run runs/E6_a4_partial`; `bash run_all.sh configs/e8_kd_a4_small.yaml`
Section ablation	`bash run_all.sh configs/a_e6_first.yaml configs/a_e6_first_last.yaml configs/a_e6_spread.yaml`
Ensemble zip	`python make_submission.py -run runs/E6_a4_partial runs/E7_a3_gradual ... -name ENS_top5`
Figures and tables	`python p4/report/make_figures.py`

To be written by the author — repository link

GitHub: `https://github.com/...`

LLM declaration

To be written by the author — LLM declaration (mandatory)

Fill in the course questionnaire. Describe honestly which parts used an LLM assistant (for example the data pipeline, model and training code, figures and this report's methods/results text) and which parts are your own (design decisions, hyperparameters, analysis, conclusions).

References

[1] E. Ben Zaken, S. Ravfogel, Y. Goldberg. BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. ACL 2022.

[2] H. Bihany, S. Patel, A. Modi. LoRMA: Low-rank multiplicative adaptation for LLMs. Findings of ACL 2025.

[3] K. Cho et al. Learning phrase representations using RNN encoder–decoder for statistical machine translation. EMNLP 2014.

[4] G. Hinton, O. Vinyals, J. Dean. Distilling the knowledge in a neural network. arXiv:1503.02531, 2015.

[5] N. Houlsby et al. Parameter-efficient transfer learning for NLP. ICML 2019.

[6] J. Howard, S. Ruder. Universal language model fine-tuning for text classification. ACL 2018.

[7] V. Malik et al. ILDC for CJPE: Indian legal documents corpus for court judgment prediction and explanation. ACL 2021.

[8] Z. Yang et al. Hierarchical attention networks for document classification. NAACL 2016.

[9] Q. Zhang et al. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. ICLR 2023.